

TUGAS PROJECT APLIKASI BASIS DATA



Aplikasi Basis Data Music Streaming Playlist

Dosen Pengampu :

Siti Ernawati S.Kom, M.Kom

Kelompok: 3	
Kelas: 19.2A.13	
Muhammad Ikhsan Basuki	(19240452)
Marcho Dominggus Deviano Timisela	(19240228)
Nico Alfianto	(19240013)
Awaludin AL Ayubi	(19240187)
Muhammad Nabil Indrawan	(19240440)
Fazar Alif Raidhat	(19240466)

PROGRAM STUDI SISTEM INFORMASI KAMPUS KOTA BOGOR

FAKULTAS TEKNIK DAN INFORMATIKA

UNIVERSITAS BINA SARANA INFORMATIKA

JULI 2025

KATA PENGANTAR

Puji syukur kami panjatkan kehadiran Allah SWT yang telah memberikan rahmat dan hidayah-Nya sehingga kami dapat menyelesaikan proyek akhir dengan tema "Aplikasi Basis Data Music Streaming Playlist" dengan baik dan tepat waktu. Laporan ini disusun sebagai bagian dari tugas akhir mata kuliah Aplikasi Basis Data di program studi Sistem Informasi, Universitas Bina Sarana Informatika.

Mengapa kita memilih tema "Aplikasi Basis Data Music Streaming Playlist" dikarenakan meningkatnya popularitas layanan musik streaming dan kebutuhan akan sistem yang efektif untuk mengelola playlist musik. Dengan adanya aplikasi basis data music streaming playlist, pengguna dapat dengan mudah mencari, membuat, dan mengelola playlist musik favorit mereka.

Tujuan utama Laporan ini adalah untuk memenuhi syarat nilai mata kuliah sekaligus memperdalam pemahaman tentang bagaimana cara kita menganalisis dan mengembangkan aplikasi basis data music streaming playlist dan memahami semua proses dari awal hingga akhir. Dalam analisis ini, kami menggunakan Entity Relationship Diagram (ERD) dan Logical Relationship Diagram (LRD) untuk menggambarkan struktur data dan hubungan antar entitas.

Spesifikasi File diperlukan dalam analisis kali ini untuk mengatur dan mengelola data, seperti data lagu, artis, album, dan playlist. Dengan demikian, kami berharap laporan ini dapat memberikan informasi yang bermanfaat bagi pembaca tentang bagaimana cara kerja aplikasi basis data music streaming playlist.

Bogor, 1 Juli 2025

DAFTAR ISI

TUGAS PROJECT APLIKASI BASIS DATA	i
KATA PENGANTAR	ii
DAFTAR ISI.....	1
BAB I PENDAHULUAN	2
1.1 Latar Belakang	2
1.2 Tujuan.....	2
1.3 Ruang Lingkup.....	2
BAB II PEMBAHASAN.....	3
2.1 Konsep Logis Database	3
2.2 Spesifikasi Tabel.....	4
2.3 Penggunaan Perintah SQL	6
BAB III PENUTUP.....	16
3.1 Kesimpulan.....	16
3.2 Saran.....	16
DAFTAR PUSTAKA	17
LAMPIRAN	18

BAB I

PENDAHULUAN

1.1 Latar Belakang

Meningkatnya popularitas layanan musik streaming telah mengubah cara orang mendengarkan musik. Platform musik streaming seperti Spotify, Apple Music, dan Tidal telah menjadi sangat populer di kalangan masyarakat (IFPI, 2022). Menurut laporan Global Music Report 2022, jumlah pengguna layanan musik streaming telah meningkat sebesar 25% dalam satu tahun terakhir, dengan lebih dari 500 juta pengguna aktif di seluruh dunia. Namun, dengan semakin banyaknya lagu dan artis yang tersedia, pengelolaan playlist musik menjadi semakin kompleks. Pengguna perlu memiliki kemampuan untuk mencari, membuat, dan mengelola playlist musik favorit mereka dengan mudah dan efektif.

Oleh karena itu, diperlukan sistem basis data yang efektif untuk mengelola playlist musik. Sistem basis data yang baik dapat membantu pengguna untuk mencari lagu dan artis dengan cepat, membuat playlist yang sesuai dengan selera mereka, dan mengelola playlist tersebut dengan mudah. Selain itu, sistem basis data yang efektif juga dapat membantu meningkatkan kinerja sistem dan mengurangi biaya operasional.

1.2 Tujuan

Tujuan dari penelitian ini adalah untuk merancang dan mengembangkan aplikasi basis data music streaming playlist yang dapat membantu pengguna mencari, membuat, dan mengelola playlist musik favorit mereka. Selain itu, penelitian ini juga bertujuan untuk memahami bagaimana cara kerja sistem basis data music streaming playlist dan bagaimana mengoptimalkan kinerja sistem tersebut.

1.3 Ruang Lingkup

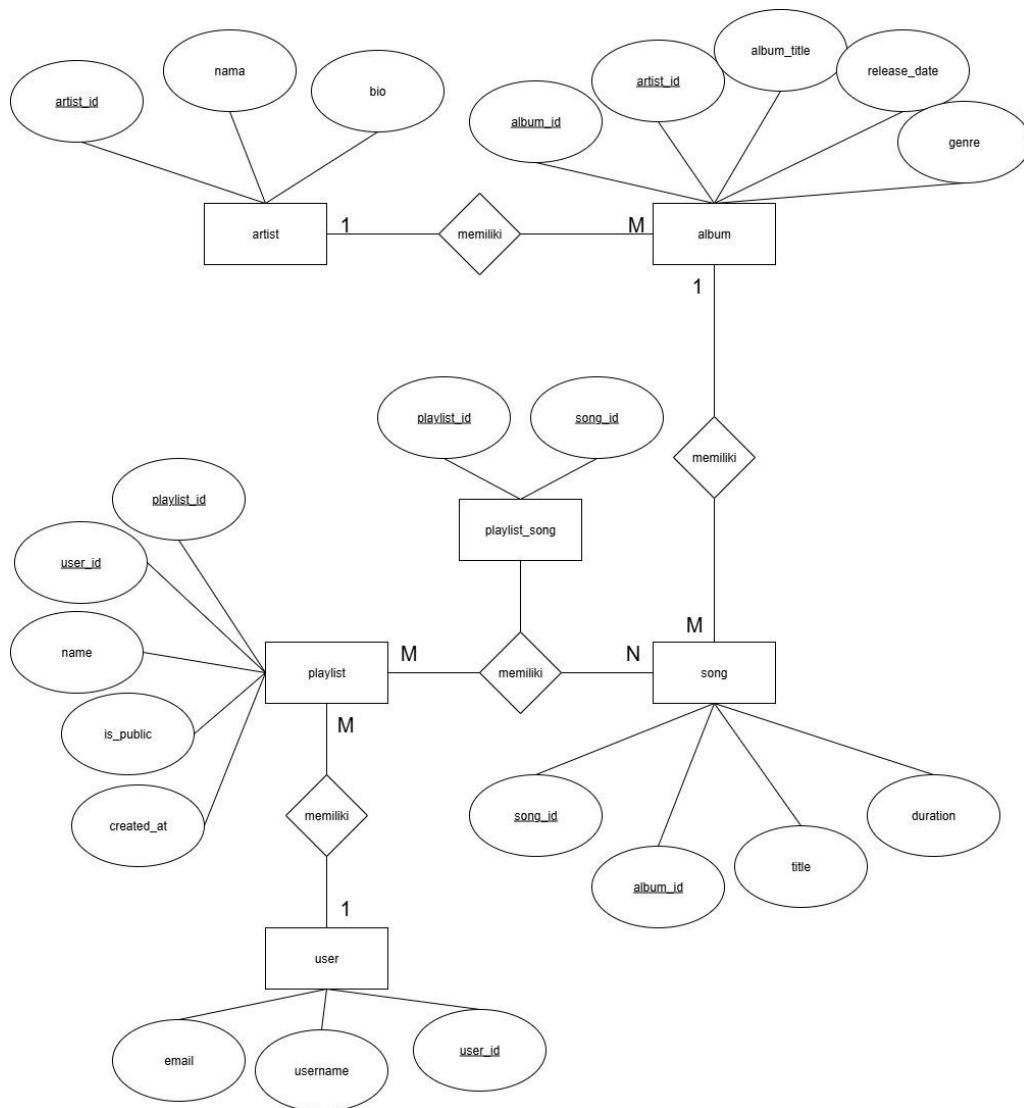
Ruang lingkup penelitian ini meliputi perancangan dan pengembangan aplikasi basis data music streaming playlist, serta analisis kinerja sistem tersebut. Penelitian ini akan menggunakan metode Entity Relationship Diagram (ERD) dan Logical Relationship Diagram (LRD) untuk menggambarkan struktur data dan hubungan antar entitas. Selain itu, penelitian ini juga akan membahas tentang spesifikasi file yang diperlukan untuk mengatur dan mengelola data lagu, artis, album, dan playlist.

BAB II PEMBAHASAN

2.1 Konsep Logis Database

ERD adalah diagram yang menggambarkan hubungan antara entitas-entitas dalam database. LRS adalah struktur logis dari database yang menggambarkan bagaimana data disimpan dan diorganisir. Berikut adalah entitas-entitas yang ada dalam database `db\_music\_stream`:

1. User: entitas yang mewakili pengguna database.
2. Artist: entitas yang mewakili artis musik.
3. Album: entitas yang mewakili album musik.

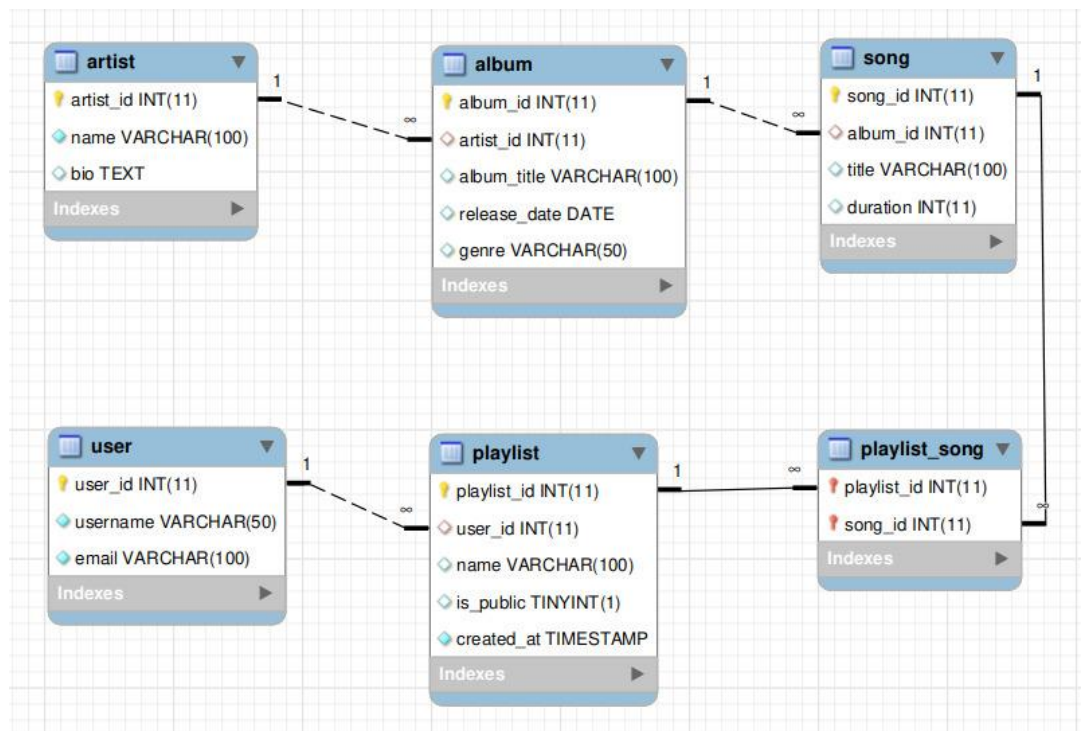


Gambar 1 Entity Relationship Diagram

4. Song: entitas yang mewakili lagu musik.
5. Playlist: entitas yang mewakili playlist musik.
6. Playlist_Song: entitas yang mewakili hubungan antara playlist dan lagu.

Hubungan antara entitas-entitas tersebut adalah sebagai berikut:

1. Sebuah User dapat memiliki banyak Playlist.
2. Sebuah Artist dapat memiliki banyak Album.
3. Sebuah Album dapat memiliki banyak Song.
4. Sebuah Playlist dapat memiliki banyak Song.
5. Sebuah Song dapat ada di banyak Playlist.



Gambar 2 Logical Record Structure

2.2 Spesifikasi Tabel

Berikut adalah spesifikasi yang ada dalam database db_music_stream:

A. Spesifikasi Tabel User

No.	Element Data	Nama <i>Field</i>	<i>Type</i>	<i>Size</i>	Keterangan
1	ID User	user_id	<i>Int</i>	11	<i>Primary Key</i> <i>Auto_increment</i>

2	Nama User	username	<i>Varchar</i>	50	
3	Email	email	<i>varchar</i>	100	

Table 1 Spesifikasi Table User

B. Spesifikasi Tabel Artist

No.	Element Data	Nama <i>Field</i>	<i>Type</i>	<i>Size</i>	Keterangan
1	ID Artist	artist_id	<i>Int</i>	11	<i>Primary key</i> <i>Auto_increment</i>
2	Nama	name	<i>Varchar</i>	100	
3	Bio	bio	<i>text</i>		

Table 2 Spesifikasi Table Artist

C. Spesifikasi Tabel Album

No.	Element Data	Nama <i>Field</i>	<i>Type</i>	<i>Size</i>	Keterangan
1	ID Album	album_id	<i>Int</i>	11	<i>Primary key</i> <i>Auto_increment</i>
2	ID Artist	artist_id	<i>int</i>	11	Foreign Key (FK)
3	Judul Album	album_title	<i>varchar</i>	100	
4	Tanggal Rilis	release_date	<i>date</i>		
5	Genre	genre	<i>varchar</i>	50	

Table 3 Spesifikasi Tabel Album

D. Spesifikasi Tabel Song

No.	Element Data	Nama <i>Field</i>	<i>Type</i>	<i>Size</i>	Keterangan
1	ID lagu	song_id	<i>Int</i>	11	<i>Primary key</i> <i>Auto_increment</i>
2	IID Album	album_id	<i>int</i>	11	Foreign Key (FK)
3	judul	title	<i>varchar</i>	100	
4	Durasi	duration	<i>Int</i>	11	

Table 4 Spesifikasi Table Song

E. Spesifikasi Tabel Playlist

No.	Element Data	Nama <i>Field</i>	<i>Type</i>	<i>Size</i>	Keterangan
-----	--------------	-------------------	-------------	-------------	------------

1	ID Playlist	playlist_id	Int	11	Primary key Auto_increment
2	ID User	user_id	Int	11	Foreign Key (FK)
3	Nama	name	varchar	100	
4	Publik	is_public	tinyint	1	
5	Dibuat	created_at	timestamp		

Table 5 Spesifikasi Table Playlist

F. Spesifikasi Tabel Playlist Song

No.	Element Data	Nama Field	Type	Size	Keterangan
1	ID Playlist	playlist_id	Int	11	Primary key Auto_increment
2	ID Lagu	song_id	Int	11	Foreign Key (FK)

Table 6 Spesifikasi Table Playlist_song

2.3 Penggunaan Perintah SQL

1. Create database dan use database :

```
MariaDB [(none)]> CREATE DATABASE db_music_stream;
Query OK, 1 row affected (0.000 sec)

MariaDB [(none)]> USE db_music_stream;
Database changed
MariaDB [db_music_stream]>
```

2. Create 6 table (user, artist, album, song, playlist, playlist_song) :

```
MariaDB [db_music_stream]> CREATE TABLE user (
->   user_id INT PRIMARY KEY AUTO_INCREMENT,
->   username VARCHAR(50) UNIQUE NOT NULL,
->   email VARCHAR(100) UNIQUE NOT NULL
-> );
Query OK, 0 rows affected (0.072 sec)
```

```
MariaDB [db_music_stream]> CREATE TABLE artist (
->   artist_id INT PRIMARY KEY AUTO_INCREMENT,
->   name VARCHAR(100) NOT NULL,
->   bio TEXT
-> );
Query OK, 0 rows affected (0.051 sec)
```



```
MariaDB [db_music_stream]> CREATE TABLE album (  
-> album_id INT PRIMARY KEY AUTO_INCREMENT,  
-> artist_id INT,  
-> title VARCHAR(100),  
-> release_date DATE,  
-> genre VARCHAR(50),  
-> FOREIGN KEY (artist_id) REFERENCES artist(artist_id)  
-> );  
Query OK, 0 rows affected (0.062 sec)
```

```
MariaDB [db_music_stream]> CREATE TABLE song (  
-> song_id INT PRIMARY KEY AUTO_INCREMENT,  
-> album_id INT,  
-> title VARCHAR(100),  
-> duration INT,  
-> FOREIGN KEY (album_id) REFERENCES album(album_id)  
-> ON DELETE CASCADE  
-> );  
Query OK, 0 rows affected (0.060 sec)
```

```
MariaDB [db_music_stream]> CREATE TABLE playlist (  
-> playlist_id INT PRIMARY KEY AUTO_INCREMENT,  
-> user_id INT,  
-> name VARCHAR(100),  
-> is_public BOOLEAN DEFAULT FALSE,  
-> created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
-> FOREIGN KEY (user_id) REFERENCES user(user_id)  
-> );  
Query OK, 0 rows affected (0.065 sec)
```

```
MariaDB [db_music_stream]> CREATE TABLE playlist_song (  
-> playlist_id INT,  
-> song_id INT,  
-> PRIMARY KEY (playlist_id, song_id),  
-> FOREIGN KEY (playlist_id) REFERENCES playlist(playlist_id),  
-> FOREIGN KEY (song_id) REFERENCES song(song_id)  
-> ON DELETE CASCADE  
-> );  
Query OK, 0 rows affected (0.064 sec)
```

3. Alter table album menjadi album_title :

```
MariaDB [db_music_stream]> ALTER TABLE album
-> CHANGE COLUMN title album_title VARCHAR(100);
Query OK, 0 rows affected (0.022 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

4. Describe 6 table (user, artist, album, song, playlist, playlist_song) :

```
MariaDB [db_music_stream]> DESC user;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| user_id | int(11) | NO | PRI | NULL | auto_increment |
| username | varchar(50) | NO | UNI | NULL | |
| email | varchar(100) | NO | UNI | NULL | |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.002 sec)
```

```
MariaDB [db_music_stream]> DESC artist;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| artist_id | int(11) | NO | PRI | NULL | auto_increment |
| name | varchar(100) | NO | | NULL | |
| bio | text | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.002 sec)
```

```
MariaDB [db_music_stream]> DESC album;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| album_id | int(11) | NO | PRI | NULL | auto_increment |
| artist_id | int(11) | YES | MUL | NULL | |
| album_title | varchar(100) | YES | | NULL | |
| release_date | date | YES | | NULL | |
| genre | varchar(50) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.002 sec)
```

```
MariaDB [db_music_stream]> DESC song;
```

Field	Type	Null	Key	Default	Extra
song_id	int(11)	NO	PRI	NULL	auto_increment
album_id	int(11)	YES	MUL	NULL	
title	varchar(100)	YES		NULL	
duration	int(11)	YES		NULL	

4 rows in set (0.002 sec)

```
MariaDB [db_music_stream]> DESC playlist;
```

Field	Type	Null	Key	Default	Extra
playlist_id	int(11)	NO	PRI	NULL	auto_increment
user_id	int(11)	YES	MUL	NULL	
name	varchar(100)	YES		NULL	
is_public	tinyint(1)	YES		0	
created_at	timestamp	NO		current_timestamp()	

5 rows in set (0.002 sec)

```
MariaDB [db_music_stream]> DESC playlist_song;
```

Field	Type	Null	Key	Default	Extra
playlist_id	int(11)	NO	PRI	NULL	
song_id	int(11)	NO	PRI	NULL	

2 rows in set (0.002 sec)

5. Insert 6 table (user, artist, album, song, playlist, playlist_song) :

```
MariaDB [db_music_stream]> INSERT INTO user (username, email) VALUES
-> ('musicfan01', 'fan01@example.com'),
-> ('beatlover', 'beatlover@example.com'),
-> ('audiophile', 'audiophile@example.com');
Query OK, 3 rows affected (0.012 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```

MariaDB [db_music_stream]> INSERT INTO artist (name, bio) VALUES
-> ('Avicii', 'Swedish DJ, remixer, record producer, and musician.'),
-> ('The Wellermen', 'Folk group known for sea shanties.'),
-> ('C418', 'German musician and composer, best known for Minecraft music.'),
-> ('John Powell', 'Film composer known for scores like How to Train Your Dragon.');
```

Query OK, 4 rows affected (0.015 sec)
Records: 4 Duplicates: 0 Warnings: 0

```

MariaDB [db_music_stream]> INSERT INTO album (artist_id, album_title, release_date, genre) VA
LUES
-> (1, 'True', '2013-09-13', 'Electronic'),
-> (1, 'Stories', '2015-10-02', 'Electronic'),
-> (2, 'Sea Shanties', '2021-01-15', 'Folk'),
-> (3, 'Minecraft - Volume Alpha', '2011-03-04', 'Ambient'),
-> (4, 'How to Train Your Dragon OST', '2010-03-23', 'Soundtrack');
```

Query OK, 5 rows affected (0.022 sec)
Records: 5 Duplicates: 0 Warnings: 0

```

MariaDB [db_music_stream]> INSERT INTO song (album_id, title, duration) VALUES
-> (1, 'Wake Me Up', 245),
-> (1, 'Hey Brother', 231),
-> (2, 'Waiting for Love', 228),
-> (3, 'Wellerman', 174),
-> (4, 'Sweden', 231),
-> (4, 'Mice on Venus', 285),
-> (5, 'This is Berk', 256),
-> (5, 'Test Drive', 221);
```

Query OK, 8 rows affected (0.040 sec)
Records: 8 Duplicates: 0 Warnings: 0

```

MariaDB [db_music_stream]> INSERT INTO playlist (user_id, name, is_public) VALUES
-> (1, 'My EDM Hits', TRUE),
-> (2, 'Chill Vibes', FALSE),
-> (3, 'Soundtrack Love', TRUE);
```

Query OK, 3 rows affected (0.010 sec)
Records: 3 Duplicates: 0 Warnings: 0

```

MariaDB [db_music_stream]> INSERT INTO playlist_song (playlist_id, song_id) VALUES
-> (1, 1),
-> (1, 2),
-> (1, 3),
-> (2, 4),
-> (2, 5),
-> (2, 6),
-> (3, 7),
-> (3, 8);
```

Query OK, 8 rows affected (0.031 sec)
Records: 8 Duplicates: 0 Warnings: 0

6. Update table :

```

MariaDB [db_music_stream]> UPDATE user
-> SET email="musicfan01@example.com"
-> WHERE user_id=1;
```

Query OK, 1 row affected (0.012 sec)
Rows matched: 1 Changed: 1 Warnings: 0

7. Delete table :

```
MariaDB [db_music_stream]> DELETE FROM playlist_song
-> WHERE playlist_id=1 AND song_id=1;
Query OK, 1 row affected (0.010 sec)
```

8. Select 6 table (user, artist, album, song, playlist, playlist_song) :

```
MariaDB [db_music_stream]> SELECT * FROM user;
+-----+-----+-----+
| user_id | username | email |
+-----+-----+-----+
| 1 | musicfan01 | musicfan01@example.com |
| 2 | beatlover | beatlover@example.com |
| 3 | audiophile | audiophile@example.com |
+-----+-----+-----+
3 rows in set (0.000 sec)
```

```
MariaDB [db_music_stream]> SELECT * FROM artist;
+-----+-----+-----+
| artist_id | name | bio |
+-----+-----+-----+
| 1 | Avicii | Swedish DJ, remixer, record producer, and musician. |
| 2 | The Wellermen | Folk group known for sea shanties. |
| 3 | C418 | German musician and composer, best known for Minecraft music. |
| 4 | John Powell | Film composer known for scores like How to Train Your Dragon. |
+-----+-----+-----+
4 rows in set (0.000 sec)
```

```
MariaDB [db_music_stream]> SELECT * FROM song;
+-----+-----+-----+-----+
| song_id | album_id | title | duration |
+-----+-----+-----+-----+
| 1 | 1 | Wake Me Up | 245 |
| 2 | 1 | Hey Brother | 231 |
| 3 | 2 | Waiting for Love | 228 |
| 4 | 3 | Wellerman | 174 |
| 5 | 4 | Sweden | 231 |
| 6 | 4 | Mice on Venus | 285 |
| 7 | 5 | This is Berk | 256 |
| 8 | 5 | Test Drive | 221 |
+-----+-----+-----+-----+
8 rows in set (0.000 sec)
```

```
MariaDB [db_music_stream]> SELECT * FROM playlist;
+-----+-----+-----+-----+-----+
| playlist_id | user_id | name | is_public | created_at |
+-----+-----+-----+-----+-----+
| 1 | 1 | My EDM Hits | 1 | 2025-07-02 09:20:56 |
| 2 | 2 | Chill Vibes | 0 | 2025-07-02 09:20:56 |
| 3 | 3 | Soundtrack Love | 1 | 2025-07-02 09:20:56 |
+-----+-----+-----+-----+-----+
3 rows in set (0.000 sec)
```

```
MariaDB [db_music_stream]> SELECT * FROM album;
```

album_id	artist_id	album_title	release_date	genre
1	1	True	2013-09-13	Electronic
2	1	Stories	2015-10-02	Electronic
3	2	Sea Shanties	2021-01-15	Folk
4	3	Minecraft - Volume Alpha	2011-03-04	Ambient
5	4	How to Train Your Dragon OST	2010-03-23	Soundtrack

```
5 rows in set (0.000 sec)
```

```
MariaDB [db_music_stream]> SELECT * FROM playlist_song;
```

playlist_id	song_id
1	2
1	3
2	4
2	5
2	6
3	7
3	8

```
7 rows in set (0.001 sec)
```

9. Select artist, album, song, dan duration :

```
MariaDB [db_music_stream]> SELECT
-> ar.name AS artist,
-> al.album_title AS album,
-> s.title AS song,
-> s.duration
-> FROM song s
-> INNER JOIN album al ON s.album_id=al.album_id
-> INNER JOIN artist ar ON al.artist_id=ar.artist_id;
```

artist	album	song	duration
Avicii	True	Wake Me Up	245
Avicii	True	Hey Brother	231
Avicii	Stories	Waiting for Love	228
The Wellermen	Sea Shanties	Wellerman	174
C418	Minecraft - Volume Alpha	Sweden	231
C418	Minecraft - Volume Alpha	Mice on Venus	285
John Powell	How to Train Your Dragon OST	This is Berk	256
John Powell	How to Train Your Dragon OST	Test Drive	221

```
8 rows in set (0.001 sec)
```

10. Create view song_v :

```
MariaDB [db_music_stream]> CREATE VIEW song_v AS
-> SELECT
->     ar.name AS artist,
->     al.album_title AS album,
->     s.title AS song,
->     s.duration
-> FROM song s
-> INNER JOIN album al ON s.album_id=al.album_id
-> INNER JOIN artist ar ON al.artist_id=ar.artist_id;
Query OK, 0 rows affected (0.010 sec)

MariaDB [db_music_stream]>
MariaDB [db_music_stream]> SELECT * FROM song_v;
+-----+-----+-----+-----+
| artist | album | song | duration |
+-----+-----+-----+-----+
| Avicii | True | Wake Me Up | 245 |
| Avicii | True | Hey Brother | 231 |
| Avicii | Stories | Waiting for Love | 228 |
| The Wellermen | Sea Shanties | Wellerman | 174 |
| C418 | Minecraft - Volume Alpha | Sweden | 231 |
| C418 | Minecraft - Volume Alpha | Mice on Venus | 285 |
| John Powell | How to Train Your Dragon OST | This is Berk | 256 |
| John Powell | How to Train Your Dragon OST | Test Drive | 221 |
+-----+-----+-----+-----+
8 rows in set (0.001 sec)
```

11. Mencari lagu yang dibikin oleh Avicii :

```
MariaDB [db_music_stream]> SELECT *
-> FROM song_v
-> WHERE artist="avicii";
+-----+-----+-----+-----+
| artist | album | song | duration |
+-----+-----+-----+-----+
| Avicii | True | Wake Me Up | 245 |
| Avicii | True | Hey Brother | 231 |
| Avicii | Stories | Waiting for Love | 228 |
+-----+-----+-----+-----+
3 rows in set (0.001 sec)
```

12. Mencari lagu dengan durasi antara 200-280 detik :

```
MariaDB [db_music_stream]> SELECT *
-> FROM song_v
-> WHERE duration>=200 AND duration<=280;
```

artist	album	song	duration
Avicii	True	Wake Me Up	245
Avicii	True	Hey Brother	231
Avicii	Stories	Waiting for Love	228
C418	Minecraft - Volume Alpha	Sweden	231
John Powell	How to Train Your Dragon OST	This is Berk	256
John Powell	How to Train Your Dragon OST	Test Drive	221

6 rows in set (0.001 sec)

13. Melihat berapa banyak lagu yang dibikin oleh masing-masing artist :

```
MariaDB [db_music_stream]> SELECT
-> ar.name AS artist,
-> COUNT(s.song_id) AS total_songs
-> FROM artist ar
-> INNER JOIN album al ON ar.artist_id=al.artist_id
-> INNER JOIN song s ON al.album_id=s.album_id
-> GROUP BY ar.artist_id
-> ORDER BY total_songs DESC;
```

artist	total_songs
Avicii	3
C418	2
John Powell	2
The Wellermen	1

4 rows in set (0.001 sec)

14. Menampilkan playlist yang dibikin oleh user :

```
MariaDB [db_music_stream]> SELECT
-> p.name AS playlist,
-> u.username AS user,
-> if(p.is_public IS TRUE,"Public","Private") AS visibility,
-> TIMESTAMPDIFF(DAY, p.created_at, NOW()) AS create_days_ago
-> FROM playlist p
-> INNER JOIN user u ON p.user_id=u.user_id;
```

playlist	user	visibility	create_days_ago
My EDM Hits	musicfan01	Public	0
Chill Vibes	beatlover	Private	0
Soundtrack Love	audiophile	Public	0

3 rows in set (0.001 sec)

15. Menampilkan lagu di dalam playlist :

```
MariaDB [db_music_stream]> SELECT
->     u.username,
->     p.name AS playlist,
->     s.title
-> FROM user u
-> INNER JOIN playlist p ON u.user_id=p.user_id
-> INNER JOIN playlist_song ps ON p.playlist_id=ps.playlist_id
-> INNER JOIN song s ON ps.song_id=s.song_id;
```

username	playlist	title
musicfan01	My EDM Hits	Hey Brother
musicfan01	My EDM Hits	Waiting for Love
beatlover	Chill Vibes	Wellerman
beatlover	Chill Vibes	Sweden
beatlover	Chill Vibes	Mice on Venus
audiophile	Soundtrack Love	This is Berk
audiophile	Soundtrack Love	Test Drive

7 rows in set (0.001 sec)

BAB III

PENUTUP

3.1 Kesimpulan

Database `db\_music\_stream` membantu pengelolaan data musik menjadi lebih terstruktur dan efisien. Sistem database memungkinkan pencatatan data pengguna, artis, album, lagu, playlist, dan hubungan antara playlist dan lagu dilakukan dengan rapi, akurat, serta mudah diakses dan dikelola. Relasi antar tabel dalam database memperjelas hubungan antar entitas, seperti artis memiliki album, album memiliki lagu, dan pengguna memiliki playlist. Dengan menggunakan Entity Relationship Diagram (ERD), hubungan seperti ini menjadi lebih mudah dipahami dan diimplementasikan.

3.2 Saran

Untuk meningkatkan kemudahan penggunaan sistem oleh pengguna, disarankan agar database `db\_music\_stream` diintegrasikan dengan antarmuka berbasis web atau mobile. Hal ini akan memungkinkan pengelolaan data yang lebih cepat dan aksesibilitas yang lebih tinggi. Selain itu, sistem keamanan perlu ditingkatkan dengan penambahan fitur otentikasi pengguna dan pembatasan hak akses sesuai peran, langkah ini penting untuk menjaga kerahasiaan data pribadi pengguna dan data transaksi. Dengan demikian, sistem database `db\_music\_stream` dapat menjadi lebih efektif dan efisien dalam mengelola data musik.

DAFTAR PUSTAKA

- IFPI. (2022). Global Music Report 2022. International Federation of the Phonographic Industry.
- Spotify. (2022). Spotify Developer Documentation. Spotify AB.
- Apple Music. (2022). Apple Music API Documentation. Apple Inc.
- Cloudwick. (2022). Cloud Computing for Music Streaming. Cloudwick Technologies.
- MobileStats. (2022). Mobile Music Streaming Statistics. MobileStats Research.
- IEEE. (2024). Design and Implementation of Personalized Online Music Streaming Media System Based on Hybrid Recommendation Algorithm. Institute of Electrical and Electronics Engineers.

LAMPIRAN

